```
object Master {

  sealed trait Event
  final case object Tick extends Event                                  ❶
  final case class CountedWords(aggregation: Map[String, Int])          ❷
      extends Event
      with CborSerializable

  final case class FailedJob(text: String) extends Event                ❸

  ...
}
```

❶ To schedule the creation of text to count

❷ To process the words grouped and counted by the worker

❸ To deal with failed jobs by adding the text back to the lag queue

At initialization, the master schedules a `Tick` message to itself every second, generating a fake text for processing. After initialization, it changes its state to `working`:

```
import akka.actor.typed.{ ActorRef, Behavior }
import akka.actor.typed.scaladsl.Behaviors

object Master {

  ...                                                                   ❶

  def apply(workerRouter: ActorRef[Worker.Command]): Behavior[Event] =
    Behaviors.withTimers { timers =>
      timers.startTimerWithFixedDelay(Tick, Tick, 1.second)             ❷
      working(workerRouter)                                             ❸
    }

   final def working ...                                                ❸

}
```

❶ Protocol

❷ Timer to emulate an event per second. The signature is (key, message, delay).

❸ Behavior after initialization